promptlock

Enterprise Product Specification • v1.0

Prompt version control, audit trails, team collaboration, and compliance — for teams that ship AI to production.

Status: Specification	Type: CLI + SaaS	Target: Enterprise AI Teams	Language: Python	Feb 2026

1. Executive Summary

AI teams at mid-to-large enterprises are managing dozens — sometimes hundreds — of prompts across models, environments, and teams. Today, those prompts live in Notion pages, Slack messages, spreadsheets, and .txt files. When a prompt breaks in production, there is no audit trail. When a new model is deployed, there is no rollback. When a compliance officer asks what changed and who approved it, there is no answer.

promptlock is a prompt version control system purpose-built for production AI teams. It gives individual developers a fast CLI interface that feels like git, and gives enterprises the governance, audit, security, and team collaboration features they require before deploying AI systems at scale.

> **CORE VALUE PROPOSITION**
> promptlock is the first prompt management tool designed from the ground up to satisfy both the developer who wants a fast CLI and the enterprise that needs SOC 2-grade audit trails, RBAC, secrets management, and SSO.

What we are building

For developers (OSS CLI)

- git-style prompt versioning: save, diff, tag, rollback
- Works in any project — no framework lock-in
- Prompt environment support (dev / staging / prod)
- Native llm-diff integration for output comparison
- Works offline; syncs to the cloud when connected

For enterprises (SaaS platform)

- Centralized prompt registry with access control
- Full immutable audit log: who changed what, when, why
- RBAC with approval workflows for prompt promotion
- SSO via SAML 2.0 / OIDC (Okta, Azure AD, Google)
- Secrets and variable management with vault integration

© 2026 promptlock • All rights reserved

2. Problem Statement

2.1 The prompt management crisis

As enterprises move from AI experimentation to production, prompts have become mission-critical software artifacts — but they are managed like sticky notes. This creates four compounding risks:

Risk Category	Current Reality	Business Impact
No version history	Prompts are edited in-place with no record of previous versions	Cannot rollback when a prompt change degrades output quality
No audit trail	No record of who changed a prompt, when, or why	Fails compliance audits; cannot investigate production incidents
No access control	Anyone with the credentials can modify production prompts	Unauthorized changes can break customer-facing AI features
No environment management	Same prompt used in dev and prod; no promotion workflow	Dev experiments accidentally reach production
No approval process	Prompt changes deploy instantly with no review gate	No quality control before AI output reaches end users
No secrets hygiene	API keys, system instructions, and sensitive variables mixed into prompt files	Credentials leak into version control; security vulnerabilities

ENTERPRISE TRIGGER

A Fortune 500 company's customer service AI gave incorrect refund policy information for 6 hours after an unauthorized prompt edit. The company had no rollback mechanism and no audit trail. This is the incident that enterprise buyers are trying to prevent.

3. Target Users & Personas

Persona	Role	Primary Need	Key Pain Today
Alex — AI Engineer	Senior engineer building LLM features	Fast CLI to version and test prompts locally	Copy-pasting prompts between Notion and code
Priya — AI Team Lead	Manages a team of 6 AI engineers	Visibility into what prompts are running in prod	No idea who changed what or when
Marcus — MLOps	Responsible for model and prompt deployments	Promotion workflows with approvals	Prompts deployed the same way as config files — no gates
Dana — Compliance Officer	Ensures AI systems meet regulatory requirements	Immutable audit logs and access records	Cannot answer auditor questions about AI system changes
Chris — CTO	Oversees AI strategy and security posture	Centralized governance without slowing teams	Shadow prompt files across 12 different teams

© 2026 promptlock • All rights reserved

4. Feature Specification

4.1 Core CLI — Prompt Version Control

The CLI is the developer-facing surface. It must feel as natural as git. Zero config required for a first-time user — one command installs, one command initializes a project.

```
# Install
pip install promptlock

# Initialize a project (creates .promptlock/ directory)
promptlock init

# Save a new version of a prompt
promptlock save prompts/summarize.txt -m "Tighten instruction, reduce
hallucinations"

# View version history
promptlock log prompts/summarize.txt

# Diff two versions
promptlock diff prompts/summarize.txt v3 v4

# Rollback to a previous version
promptlock rollback prompts/summarize.txt v3

# Tag a version for promotion
promptlock tag prompts/summarize.txt v4 --name stable-2026-02

# Push to team registry (requires auth)
promptlock push
```

4.2 Prompt Registry — Centralized Store

The registry is the single source of truth for all prompts across an organization. It is the backend that the CLI syncs to and the web dashboard queries from.

- Hierarchical namespace: org / team / project / prompt
- Immutable storage — versions are never overwritten, only appended
- Content-addressed storage: each version has a SHA-256 hash for integrity verification
- Metadata on every version: author, timestamp, commit message, model target, environment, parent version
- Prompt search: full-text search across all prompt content and metadata
- Prompt linking: track which prompts depend on or are derived from others

> **ENTERPRISE REQUIREMENT**
> Immutable storage is non-negotiable for compliance. Once a prompt version is saved, it cannot be modified or deleted by any user — including administrators. Deletion requests must go through a retention policy workflow with approval.

4.3 Access Control — RBAC

Role-based access control governs who can read, write, approve, and delete prompts at every level of the hierarchy.

Role	Scope	Permissions
Viewer	Org / Team / Project	Read prompt content and history. Cannot save new versions.
Contributor	Team / Project	Save new versions in non-production environments. Cannot promote to prod.
Reviewer	Team / Project	Review and approve/reject promotion requests. Cannot self-approve.
Deployer	Project	Promote approved versions to production. Cannot approve own requests.
Admin	Team	Manage team members, roles, and project settings. Cannot modify audit logs.
Org Admin	Org	Manage SSO, billing, org-wide policies, retention rules. Full read access.
Auditor	Org	Read-only access to all audit logs across the entire org. Cannot modify anything.

- Roles are additive and assigned at the most specific scope possible
- Separation of duties enforced: no user can both create a version and approve it for production
- Service accounts supported for CI/CD pipelines with scoped API tokens
- Emergency access ('break-glass') with mandatory audit notification

4.4 Approval Workflows — Promotion Gates

Promotion workflows enforce a mandatory review process before any prompt change reaches a higher environment. This is the key governance feature that enterprise compliance teams require.

Workflow Step	Actor	System Action	Audit Event
1. Save draft	Contributor	New version created in dev environment	PROMPT_VERSION_CREATED
2. Run eval (optional)	Automated / CI	llm-diff or evalkit checks pass	EVAL_RUN_COMPLETED
3. Submit for review	Contributor	Promotion request created; reviewers notified	PROMOTION_REQUESTED
4. Review	Reviewer	Diff shown; reviewer adds comments	REVIEW_STARTED
5. Approve / Reject	Reviewer	Decision recorded with mandatory comment	PROMOTION_APPROVED / REJECTED
6. Promote	Deployer	Version activated in target environment	PROMPT_PROMOTED
7. Monitor	System	Output quality metrics tracked post-deploy	DEPLOYMENT_MONITORED

© 2026 promptlock • All rights reserved

8. Rollback (if needed)	Deployer	Previous version reactivated instantly	PROMPT_ROLLED_BACK

- Configurable required approver count (default: 1, enterprise: 2+)
- Time-boxed approvals: auto-escalate if no action within configurable window
- Integration with Slack and email for review notifications
- Approval bypass requires Org Admin role and generates high-severity audit event

4.5 Audit Log — Immutable Compliance Trail

Every action in promptlock — read, write, approve, login, export — is recorded in an immutable, append-only audit log. The log is the primary artifact for compliance audits, security investigations, and incident post-mortems.

Audit event schema

```
{ "event_id": "evt_01HX...",
  "timestamp": "2026-02-15T14:32:01.442Z",   // UTC, microsecond precision
  "event_type": "PROMPT_PROMOTED",
  "actor": { "user_id": "usr_abc", "email": "priya@acme.com", "ip":
"203.0.113.5" },
  "resource": { "type": "prompt", "id": "pmt_xyz", "version": "v7" },
  "org_id": "org_123",  "team_id": "team_456",
  "metadata": { "environment": "production", "approver": "usr_def" },
  "checksum": "sha256:a3f9..." }  // tamper-evident hash
```

Enterprise log requirements

- Append-only: no row can be modified or deleted by any actor including Org Admins
- Tamper-evident: each log entry includes a hash chained to the previous entry
- Retention policy: configurable minimum retention (default 2 years; up to 7 years for regulated industries)
- Export: full log export to SIEM systems (Splunk, Datadog, Elastic) via webhook or scheduled export
- Search and filter: queryable by actor, event type, resource, date range, and environment
- Real-time alerting: configurable alerts on high-severity events (bypass, mass export, failed login attempts)

4.6 Environment Management

Prompts exist across multiple environments — development, staging, and production — with explicit promotion gates between them. Each environment has independent configuration and access controls.

Environment	Who Can Write	Who Can Read	Promotion Gate
development	Contributors+	All team members	None — push freely
staging	Deployers (via promotion)	All team members + CI	1 Reviewer approval required
production	Deployers (via promotion)	Defined read-access list	2 Reviewer approvals + Deployer action

archived	No one	Auditors + Admins	Automatic after retention period

- Custom environments supported (e.g., canary, shadow, a-b-test)
- Environment variables and secrets are scoped per environment — dev and prod secrets never mix
- Environment-specific model configuration: a prompt can target different models per environment

4.7 Secrets & Variable Management

Prompts often contain sensitive instructions, system personas, API references, or customer-specific variables. promptlock treats these as first-class secrets — never stored in plaintext alongside prompt content.

- **Variable templating:** Use {{variable_name}} syntax in prompt files; values injected at render time
- **Secret storage:** Values encrypted at rest with AES-256; keys managed per org
- **Vault integration:** Native connectors for HashiCorp Vault, AWS Secrets Manager, Azure Key Vault
- **Secret scanning:** Automatically detects and blocks accidental commit of API keys or credentials
- **Access scoping:** Secrets accessible only to the environments and roles explicitly granted
- **Secret rotation:** Rotation events recorded in audit log; old values remain in versioned history but are flagged

4.8 SSO & Identity Management

Enterprise authentication is mandatory for teams managing production AI systems. promptlock supports industry-standard identity federation with no vendor lock-in.

Feature	Details
SAML 2.0	IdP-initiated and SP-initiated SSO. Tested with Okta, Azure AD, Ping Identity, OneLogin
OIDC / OAuth 2.0	Google Workspace, GitHub Enterprise, custom OIDC providers
SCIM 2.0 provisioning	Automated user provisioning and deprovisioning from IdP. Groups sync to teams.
MFA enforcement	Org Admins can require MFA for all users. TOTP and WebAuthn (hardware key) supported.
Session management	Configurable session duration. Force re-auth for sensitive actions (promote to prod).
Service account tokens	Short-lived JWT tokens for CI/CD. Scoped to specific projects and permissions.
IP allowlisting	Restrict access to corporate IP ranges per org or team.

4.9 CI/CD Integration

promptlock is designed to live inside existing engineering workflows. Prompt changes should go through the same review and deployment pipeline as code changes.

```
# GitHub Actions example
- name: Validate prompts
  run: |
    promptlock validate --env staging
    promptlock diff --base main --head HEAD --format json | jq .

# Promote on merge to main
- name: Promote prompts to staging
  run: promptlock promote --from dev --to staging --auto-approve
  env:
    PROMPTLOCK_TOKEN: ${{ secrets.PROMPTLOCK_CI_TOKEN }}
```

- Native GitHub Actions and GitLab CI actions published
- Prompt diff as PR comment: automatically posts a diff of changed prompts on every pull request
- Promotion status checks: block merges if prompts have unapproved staging changes
- Webhook events: trigger external systems on promotion, rollback, or approval events
- OpenTelemetry traces: emit spans for all CLI and API operations for observability integration

4.10 Web Dashboard

The dashboard gives non-CLI users (team leads, compliance officers, product managers) full visibility into the prompt registry without requiring terminal access.

- Prompt browser: searchable, filterable view of all prompts by team, project, environment, and status
- Version timeline: visual history of all versions with author, message, and diff on click
- Approval queue: reviewers see all pending promotion requests with one-click approve/reject
- Audit log viewer: searchable audit trail with export to CSV
- Team management: invite members, assign roles, manage SSO configuration
- Usage analytics: which prompts are most active, who is making the most changes, environment activity heatmap
- Alert configuration: set up real-time alerts for audit events, failed deployments, and unusual activity

4.11 llm-diff Integration

promptlock has a first-class integration with llm-diff for output comparison as part of the review workflow.

- Automatic diff on promotion request: when a promotion is submitted, promptlock can optionally run llm-diff against the previous active version and attach the output diff to the review
- Eval gates: configurable minimum similarity threshold — block promotion if semantic similarity drops below e.g. 70%
- Side-by-side in dashboard: HTML diff report embedded directly in the approval UI
- CLI shorthand: promptlock eval <prompt-file> --compare-env staging runs llm-diff automatically

5. Enterprise Compliance & Security

5.1 Security architecture

Layer	Implementation
Encryption at rest	AES-256-GCM for all prompt content, secrets, and metadata. Keys managed per org in AWS KMS or customer-managed key (BYOK).
Encryption in transit	TLS 1.3 enforced on all connections. HSTS preloaded. Certificate pinning for CLI.
Data isolation	Each org's data stored in logically isolated schema. Enterprise tier: dedicated database instance.
Secrets management	Secrets never stored alongside prompt text. Separate encrypted vault per org. Secrets not returned in API responses — only injected at render time.
Vulnerability management	Dependency scanning on every build (Snyk). Penetration test annually. Bug bounty program.
Network security	SOC 2-compliant cloud infrastructure. Private VPC. No public database endpoints. WAF on all API endpoints.

5.2 Compliance certifications (roadmap)

Standard	Target Date	Notes
SOC 2 Type I	v1.0 launch	Controls documented and tested at a point in time
SOC 2 Type II	6 months post-launch	12-month observation period of operating effectiveness
ISO 27001	12 months post-launch	Information security management system certification
GDPR compliance	v1.0 launch	Data processing agreements available; EU data residency option
HIPAA BAA available	v1.2	For healthcare customers handling PHI in prompt context
FedRAMP (future)	18+ months	For US government and regulated contractor customers

5.3 Data residency & privacy

- EU data residency: all org data stored and processed in EU-West (AWS eu-west-1) upon request
- Data processing agreement (DPA) available for all paid plans
- No model training on customer prompt data — ever. Contractually guaranteed.
- Right to deletion: org data fully purged within 30 days of account closure (audit logs retained per legal hold if applicable)
- Subprocessor list published and maintained; customers notified 30 days before new subprocessors are added

6. Technical Architecture

© 2026 promptlock • All rights reserved

6.1 System overview

CLI (Python)

- Click-based CLI, installable via pip
- Local .promptlock/ directory stores working state
- Syncs to registry via REST API with JWT auth
- Works fully offline; push/pull when connected
- Config: .promptlock.toml per project or ~/.promptlock/config

Registry API

- FastAPI (Python) REST API
- PostgreSQL for metadata and version history
- S3-compatible object storage for prompt content
- Redis for session cache and job queues
- Celery workers for async tasks (eval runs, notifications)

Web Dashboard

- Next.js frontend, deployed on Vercel CDN
- Reads from the same REST API as the CLI
- Real-time updates via WebSocket for approval queue
- No client-side secrets storage — JWT in HttpOnly cookie only

Infrastructure

- AWS: ECS Fargate for API, RDS Postgres, S3, ElastiCache
- Terraform-managed infrastructure (IaC)
- Multi-region active-passive for enterprise tier
- 99.9% uptime SLA (standard); 99.95% (enterprise)
- Automated backups: continuous WAL archiving + daily snapshots

6.2 API design

The REST API is the integration surface for CI/CD, enterprise tooling, and third-party integrations. It is versioned, documented (OpenAPI 3.1), and designed for stability.

```
# Core resource endpoints
GET    /v1/prompts                    # List prompts (paginated, filterable)
POST   /v1/prompts                    # Create a new prompt
GET    /v1/prompts/{id}/versions      # List all versions
POST   /v1/prompts/{id}/versions      # Save new version
GET    /v1/prompts/{id}/versions/{v}  # Get specific version
POST   /v1/promotions                 # Submit promotion request
PATCH  /v1/promotions/{id}            # Approve / reject
GET    /v1/audit                      # Query audit log
GET    /v1/audit/export               # Export audit log (CSV / JSON)
```

- Rate limiting: per-token limits with configurable enterprise overrides
- Pagination: cursor-based for all list endpoints
- Idempotency keys supported on all write operations
- Webhooks: configurable HTTPS webhooks for all audit events with HMAC-SHA256 signatures

7. Pricing & Packaging

Feature	OSS (Free)	Pro ($29/mo)	Enterprise (Custom)
CLI — core versioning	Unlimited	Unlimited	Unlimited
Team registry	Local only	Up to 5 users	Unlimited users
Environments	dev only	dev + staging + prod	Custom environments
Audit log	None	30-day retention	7-year retention
RBAC	None	Basic (owner/member)	Full RBAC + Auditor role
Approval workflows	None	1 required approver	Configurable (2+)
SSO (SAML / OIDC)	None	None	Required — full SCIM
Secrets management	Local .env	Encrypted vault	BYOK + Vault integration
CI/CD actions	Community	Official actions	Priority support + SLA
Data residency	US only	US only	EU / US / custom
SLA	None	None	99.95% uptime + dedicated CSM
SOC 2 report	None	Available on request	Full access
HIPAA BAA	No	No	Available (v1.2)

MONETIZATION NOTE

OSS drives adoption and community. Pro captures indie developers and small teams. Enterprise is where the revenue lives — a single Fortune 500 customer at $50K ARR pays for 3+ months of runway. Target 5 enterprise design partners before GA launch.

8. Release Roadmap

Phase	Timeline	Milestone	Enterprise Features Included
v0.1 — OSS CLI	Weeks 1–2	Core git-style versioning: save, log, diff, rollback, tag	None — developer-only
v0.2 — Registry	Weeks 3–4	Cloud sync, team sharing, basic user accounts	None — still OSS-first
v0.3 — Environments	Weeks 5–6	dev / staging / prod environments, promotion CLI	Manual promotion with no approval gate
v0.4 — Audit & RBAC	Weeks 7–9	Immutable audit log, role-based access control	Core enterprise requirements met
v0.5 — Workflows	Weeks 10–12	Approval workflows, PR diff comments, CI/CD actions	Full promotion governance
v1.0 — GA	Month 4	SSO, secrets management, dashboard, SOC 2 Type I	Full enterprise feature set

© 2026 promptlock • All rights reserved

v1.1	Month 5–6	llm-diff eval gates, SCIM, advanced alerting	Eval-gated promotions
v1.2	Month 7–9	HIPAA BAA, EU data residency, FedRAMP readiness	Regulated industry support

SCOPE RULE
Enterprise features (SSO, RBAC, audit log) must be designed correctly from v0.4 — retrofitting access control and audit trails onto an existing system is extremely painful. Get the data model right before v1.0.

9. Non-Functional Requirements

Performance

- CLI cold-start (no network): < 150ms
- Save/push a prompt version: < 500ms p95
- List 1,000 versions: < 200ms API response
- Audit log query (30-day window): < 1s
- Dashboard initial load: < 2s on 10Mbps

Scalability

- Support orgs with 10,000+ prompts and 1M+ versions
- Audit log: write-optimized for 10M+ events per org
- Horizontal scaling: API stateless; DB connection pooling via pgBouncer
- Object storage: no practical limit on prompt content size

Reliability

- API availability: 99.95% (enterprise)
- Registry writes are durable — no data loss on API crash
- CLI works fully offline; retries push with exponential backoff
- RTO: 1 hour | RPO: 5 minutes (enterprise tier)

Observability

- Structured JSON logs on all API requests
- Prometheus metrics: request latency, error rates, queue depth
- Distributed tracing: OpenTelemetry with Datadog / Grafana Tempo
- Alerting: PagerDuty integration for on-call rotation

10. Risk Register

Risk	Severity	Likelihood	Mitigation
GitHub / Anthropic ship native prompt versioning	High	Medium	Differentiate on compliance depth (RBAC, audit, SSO) — big labs will never prioritize enterprise governance
Enterprise sales cycle too long to sustain OSS momentum	High	Medium	Target 5 design partners pre-launch who co-build and provide case studies. Land and expand.

© 2026 promptlock • All rights reserved

SOC 2 Type II takes longer than 12 months	Medium	Low	Start controls documentation at v0.4. Engage auditor early.
OSS community forks the project to avoid SaaS	Low	Low	Keep core CLI MIT licensed. Enterprise features (SSO, RBAC, audit) in SaaS only — not in OSS.
Security breach exposing customer prompt content	Critical	Low	BYOK, encryption at rest, penetration tests, bug bounty. Incident response plan ready at v1.0.
Prompt storage costs scale faster than revenue	Medium	Medium	Content-addressed deduplication — identical prompt text stored once. Compress large prompts at rest.

11. Success Metrics

Metric	Target at v1.0	Target at 6 Months
GitHub stars	500+	2,000+
PyPI downloads / month	1,000+	10,000+
Enterprise design partners	5 (pre-GA)	10+
Enterprise ARR	$0 (design partners pay nothing)	$250K+
Audit log events stored	—	100M+
Mean time to rollback (enterprise)	< 30 seconds	< 30 seconds
SOC 2 Type I complete	At GA	—
NPS (developer users)	50+	60+

12. Out of Scope for v1.0

DISCIPLINE NOTE
Every item below is a reasonable future feature. None of them ship before v1.0. Adding scope before the core is solid will result in shipping nothing.

- Prompt execution / LLM API proxy (promptlock stores prompts, does not run them)
- Fine-tuning dataset management
- Multi-modal prompt support (image, audio)
- Real-time collaboration (simultaneous editing)
- VS Code or IDE extensions
- Self-hosted / on-premises deployment (v2 roadmap item)
- Native integrations with LangChain, LlamaIndex (community-built)
- Automated prompt optimization / suggestions
- FedRAMP certification (18+ months out)

© 2026 promptlock • All rights reserved

13. Open Questions

❓ Data model: files vs. named prompts

Should prompts be tracked as file paths (like git) or as named resources with a registry ID? File-based is more familiar to developers. Registry-based is cleaner for the API. Recommendation: support both — files in local mode, registry IDs in team mode.

❓ Encryption key management strategy

Should the default be Anthropic-managed keys (simpler) or customer-managed keys (BYOK) from day one? BYOK is a hard enterprise requirement but adds significant infrastructure complexity. Recommendation: Anthropic-managed at v1.0, BYOK at v1.1 with AWS KMS.

❓ Audit log storage backend

PostgreSQL append-only partition vs. dedicated immutable store (e.g., Amazon QLDB, Immudb). QLDB provides cryptographic proof of immutability out of the box. Postgres requires application-layer enforcement. Recommendation: evaluate QLDB cost — if prohibitive, use Postgres with WAL archiving and third-party tamper detection.

❓ OSS license strategy

MIT for the CLI core vs. Business Source License (BUSL) to prevent commercial forks. BUSL converts to Apache 2 after 4 years. Recommendation: MIT for CLI, proprietary for the SaaS platform. Enterprise features (SSO, RBAC, audit) not in the OSS repo.

❓ Self-hosted enterprise offering

Will enterprise customers require on-premises or VPC deployment? Many regulated industries (finance, healthcare) cannot use multi-tenant SaaS. This significantly increases engineering and support complexity. Recommendation: defer to v2.0 but design the architecture to support it.

© 2026 promptlock • All rights reserved